

When Grounding Hurts: Retrieval and Validation for LLM Repair of Quantum Programs^{*}

Xuan-Bach Le^{1,2}, Phat T. Tran-Truong^{1,2}, and Duy Nguyen³

¹ Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), Vietnam

² Vietnam National University Ho Chi Minh City (VNU-HCM), Vietnam

³ University of Economics Ho Chi Minh City (UEH), Vietnam

lexuanbach@hcmut.edu.vn, phatddd@hcmut.edu.vn,
duynguyen.726202321440@st.ueh.edu.vn

Abstract. Quantum programs exhibit failures that classical static analysers often miss. API migrations may break execution, whereas an incorrect gate or reversed qubit order can silently corrupt an otherwise valid circuit. A common engineering response is to ground a large language model (LLM) with static-analysis findings and retrieved documentation, then validate its patches by execution. Whether additional grounding consistently improves repair, however, remains unclear. We present LINTREPAIR-RAG-Q, which combines a static linter, a retrieval-grounded LLM detector, and an execution-validated repair loop. We evaluate it on Q-DEFECTS40, a reproducible benchmark of 40 Qiskit programs from eight documented defect families, using eighteen proprietary and open-weight repair backbones (open weights 3B–72B). Grounding improves detection recall from 0.72 to 0.81 with no false positives on the benchmark. For repair, its effect depends on model capability. Although grounding improves pooled accuracy by 16 points ($p < 0.001$), it raises GPT-4o-mini from 0% to 63% and lowers Sonnet from 96.9% to 87.5% (descriptive at $n=32$, corroborated on a powered synthetic set). Open-weight models exhibit the same crossover, and neutral-background controls are consistent with the injected context, rather than the analyser’s verdict alone, driving the regression. A two-factor framework organises these results and supports a capability-gated policy: ground and validate lower-capability models, but avoid presenting analyser-cleanliness claims to higher-capability models. We release Q-DEFECTS40, LINTREPAIR-RAG-Q, and scripts that regenerate every reported result from a cached run.

Keywords: Quantum software engineering · Program repair · Static analysis · Large language models · Retrieval-augmented generation · Capability-dependent evaluation.

^{*} Artifact (Q-DEFECTS40 benchmark, LINTREPAIR-RAG-Q tool, and offline reproduction scripts) and an extended version of this paper, with the formal model, the validator-loop analysis, and complete per-model results: <https://github.com/lexuanbach/lintrepair-ragq-artifact>

```

qc = QuantumCircuit(2, 2)
qc.x(0) # set qubit 0 to |1>, qubit 1 stays |0>
qc.measure([0, 1], [0, 1])
counts = run(qc) # Qiskit returns {'01': 1000}
exp = str(1) + str(0) # WRONG: builds '10' (qubit 0 read first)
# FIX: exp = str(0) + str(1) -> '01' (Qiskit prints qubit 0 last)
hits = counts.get(exp, 0) # 0 instead of 1000 -- silently wrong

```

Fig. 1. A silent endianness defect. The circuit is correct and raises no exception, yet reads the wrong qubit: Qiskit labels bitstrings *little-endian* (qubit 0 rightmost). No rule-based analyser flags it; a capable LLM fixes it unprompted *unless* first told the analyser found “no defects.”

1 Introduction

Developers build quantum software on rapidly evolving SDKs such as Qiskit, Cirq, and PennyLane, where failures often fall outside the scope of classical software-engineering tools. Some failures are explicit: Qiskit 1.0 removed the top-level `execute()` helper and moved `Aer`, breaking existing code. Others are silent: an incorrect gate, a swapped control and target, or an endianness error may return a plausible but incorrect distribution without raising an exception. Empirical studies report both classes of failure and identify API churn as a major source of developer difficulty [9,5,41,42].

Prior work has introduced quantum-specific testing [36,24], test generation [38], static analysis with LintQ [25], and program repair [34]. These tools address important but limited defect classes; rule-based analysers cannot detect silent semantic errors without a syntactic signature. Large language models (LLMs) may provide broader coverage. A common pipeline runs a static analyser, retrieves API and defect documentation, supplies both to an LLM, and validates the proposed patch by execution. Such grounding is generally expected to improve both accuracy and reliability.

This assumption has not been tested for quantum program repair across models of different capabilities. We evaluate eighteen proprietary and open-weight LLMs (3B–72B) and identify a capability-dependent crossover: grounding improves weak models but degrades strong ones. For weak models, retrieved context supplies missing knowledge of the current SDK. For strong models, the same context can interfere with independent verification and suppress repairs of silent defects that the model would otherwise make. We refer to this effect as *grounding interference*, or *anchoring*, a name for the observed regression rather than for an isolated mechanism, since the controls in §6.4 do not separate report framing from the added context itself. Although context distraction is established [30,17,29], its capability-dependent effect within a grounded repair pipeline has not been studied. We also show that prompt-based mitigation must be evaluated against a generic control; otherwise, naming defect families can leak the benchmark taxonomy. This risk extends beyond quantum software whenever an incomplete analyser informs an LLM-based repair system.

Motivating example. Figure 1 illustrates the problem: a program that reads a measurement bitstring in the wrong endianness runs, raises nothing, and returns

a silently wrong count. No rule-based linter flags it (every API call is valid), and a capable LLM given only the program fixes it. But when the prompt also carries a static report of “no structural defects” plus retrieved defect-knowledge entries, the same model may leave the bug unchanged. We measure this reversal across eighteen models and turn it into deployment guidance.

Our approach. We make this concrete with LINTREPAIR-RAG-Q, a toolchain combining (i) a static linter for structural/API defects, (ii) a retrieval-grounded LLM detector, and (iii) a repair loop that conditions on detector findings and retrieved knowledge and validates each patch by execution. We evaluate it on Q-DEFECTS40, a reproducible benchmark of 40 Qiskit 2.x programs across eight documented defect families, each paired with a human fix and an execution-based semantic oracle. Every number is produced by a script over real executions; we report exact paired tests (McNemar) alongside every interval.

Contributions. We contribute (i) LINTREPAIR-RAG-Q, a detection-and-repair toolchain unifying static analysis, retrieval-grounded LLM detection, and an execution-validated repair loop, with a descriptive two-factor framework (§3, 4); (ii) Q-DEFECTS40, a reproducible 40-program Qiskit benchmark with an execution-based semantic oracle over eight defect families (§5); and (iii) evidence across eighteen models of a capability-dependent crossover, the capability-dependence of prompt-level mitigation, and a taxonomy-leakage control on debiasing, yielding deployment guidance keyed to a cheap capability proxy ($\hat{\rho}$, the unguided single-shot run-rate; §6). The RQ answers below give the headline numbers.

Research questions and answers.

RQ1 Does retrieval-grounded LLM detection complement static analysis for quantum defects, and at what precision?

A1. Yes. The static $\cup$ LLM union raises defect recall $0.72 \rightarrow 0.81$ at 1.000 precision (zero false positives on 40 negatives).

RQ2 How does grounding a repair LLM affect repair across models of differing capability?

A2. As a capability-governed *crossover*: grounding rescues weak models but anchors strong ones. The grounding $\times$ capability interaction (slope -0.47 , crossing $\hat{\rho} \approx 0.84$), not the pooled $+16.0$, is the effect.

RQ3 Is the anchoring on strong models fixable by prompt debiasing, or is any recovery an artifact?

A3. Only conditionally. Debiasing is *itself* capability-governed; the variant that edges ahead of a generic control does so only by leaking the benchmark’s defect taxonomy, and even it does not significantly beat naive grounding.

RQ4 When does the execution-validator loop add value, as a function of base-model reliability?

A4. Its value scales with $1 - \hat{\rho}$: near-inert on strong models, materially useful as the base model weakens.

2 Background and Related Work

We assume familiarity with the basics of quantum computing: qubits and superposition, entanglement, unitary gates and circuits, and measurement in the computational basis, for which standard textbooks provide background [22,14,28]. We review only the concepts our detection-and-repair study depends on and introduce notation where it first arises.

2.1 Defects in quantum software

Quantum programs are classical host programs (here, Python) that build and run circuits through an SDK. Two properties complicate their development. First, outputs are *probabilistic*: a program returns a distribution over measurement bit-strings, so an incorrect circuit can still produce a plausible-looking result. Second, SDKs evolve rapidly: Qiskit’s 1.0 release removed top-level `execute()`, moved the `Aer` object, and removed `bind_parameters` in favour of `assign_parameters`; programs written against those interfaces need migration. Empirical studies document compatibility, integration, and quantum-specific correctness challenges [9,5,41], and benchmarks such as Bugs4Q catalogue real Qiskit faults [42]. A recurring example is *endianness*: Qiskit prints counts with qubit 0 as the rightmost character, so post-processing that indexes left-to-right reads the wrong qubit.

2.2 Testing, analysis, and repair for quantum software

Testing is the most mature line of work. QDiff applies differential testing across quantum stacks [36]; MorphQ brings metamorphic testing to Qiskit [24]; QuraTest generates quantum-aware tests [38]; and QuanTest targets adversarial robustness of quantum neural networks [31]. Surveys consolidate this area and repeatedly flag weak test oracles and the absence of broad, executable benchmarks as open problems [21,26]. For static analysis, LintQ analyses Qiskit programs and detects a fixed catalogue of structural anti-patterns at high precision [25]; by construction, such analysers cannot detect defects with no syntactic signature. A complementary, *formal* line reasons about quantum programs deductively: quantum separation logic gives a program logic for *local* reasoning about quantum code [15], offering correctness guarantees at the cost of manual specification and expert effort. A related thread pairs such formal machinery with LLMs and model checking, using refinement calculus to guide and verify LLM-generated code [2] and tools such as PAT for rigorous verification [33]. We instead target automated, execution-checked repair usable without proofs or specifications. For repair, HornBro automates patch synthesis for quantum programs [34]. Our static linter (§3) is a deliberately LintQ-style rule layer; we treat it as the rule-based component to be *combined* with, not replaced by, an LLM.

2.3 LLMs, grounding, and repair

LLMs are increasingly applied to code understanding and repair [37], usually evaluated with the $\text{pass}@k$ metric [4]. The dominant design pattern *grounds* the

model: retrieval-augmented generation (RAG) injects external knowledge [16], static-analysis findings steer attention, and execution feedback validates outputs. In the quantum setting, early work benchmarks LLMs for quantum program debugging [27] on standardized representations such as OpenQASM 3 [7], and recent comparative studies find that LLM-based linting can match or exceed rule-based LintQ, improving localisation and explanation quality [32] and lifting F1 from 0.41 (rules) to 0.68–0.70 with chain-of-thought and RAG variants [3]. These results motivate *combining* rules and LLMs, which we confirm for detection.

Repair results differ. ChatGPT fixed 29/38 Bugs4Q bugs through multi-round interaction [10], and mutation-grounded prompting lifted GPT-5 repair on a Bugs4Q subset [40]; both report that grounding helps, echoing the prevailing assumption that more grounding is strictly better. Our capability-conditional result qualifies that assumption. The NLP community has independently shown that LLMs are swayed by input framing: they exhibit *sycophancy*, deferring to assertions in the prompt [29], and produce unfaithful rationales that follow injected cues [35].

Our results also sit within a broader automated-program-repair tradition that pairs generate-and-validate loops with test or static-analysis feedback [6,1,13] and reports pass@ k for generation diversity. Our work complements this machinery and evidence that self-repair quality is itself capability-dependent and can even degrade [23,11]. Retrieval-augmented APR (RelRepair [18], ReAPR [19]) finds retrieval helpful because it supplies project-local context, and SOSecure [20] frames retrieved discussion as non-prescriptive *guidance* rather than instruction. Our capability-dependent sign flip refines these: grounding that supplies missing knowledge to weak models can distract strong ones. Our controlled dissection (§6.4) finds both the injected context and the “no defects” assertion contribute, in proportions that vary by model; the cleanliness assertion is contributing but not isolable. Mechanistically, the strong-model harm is the documented context-distraction failure of LLMs [30,17,8,39] surfacing in a static-analysis-grounded *repair* pipeline; our contribution is to probe it with neutral-background controls and show it tracks both defect type and model competence, reversing the sign of grounding’s value across eighteen models rather than acting as a uniform hazard.

Our source-level, reference-free setting differs from prior quantum software engineering tools in its input/output contract. Differential and metamorphic testing (QDiff, MorphQ [36,24]) and LintQ [25] *find* defects but do not repair; LLM-detection work [32,3] stops at detection; and HornBro [34] repairs at the circuit level but requires a *correct* reference circuit, which is ill-posed in our reference-free, source-level setting.

3 Approach: LintRepair-RAG-Q

Key idea. Detection and repair require different forms of grounding. For detection, we combine a high-precision static analyser with a retrieval-grounded LLM detector to cover complementary defect classes. For repair, we use retrieved

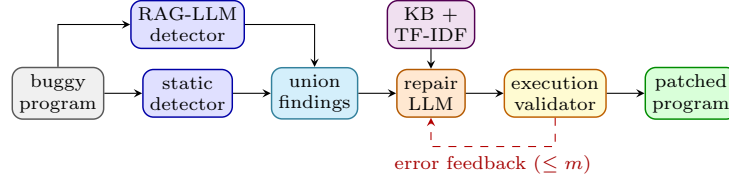


Fig. 2. LINTREPAIR-RAG-Q: a static detector and a retrieval-grounded LLM detector are unioned into findings; a repair LLM conditioned on findings and retrieved knowledge proposes patches that an execution validator accepts or returns for revision.

knowledge and execution feedback while controlling how analyser findings are presented, because an incomplete cleanliness report may suppress correct repairs.

3.1 Architecture

LINTREPAIR-RAG-Q has three components (Fig. 2).

(1) Static detector. An AST- and pattern-level analyser, in the spirit of LintQ [25], that flags structural and API defect families it can soundly recognise: removed/renamed APIs (`execute`, `Aer`, `bind_parameters`), out-of-range qubit indices, classical-register size mismatches, and counts requested without measurement. It never executes code and, by design, is silent on defects with no syntactic signature.

(2) Retrieval-grounded LLM detector. A curated knowledge base of *general* defect-family descriptions and remedies (drawn from SDK migration guides and the empirical literature) is indexed by a TF-IDF retriever. No knowledge-base entry encodes the answer to a benchmark instance. Each describes a family in the form of developer documentation, so retrieval supplies background rather than labels. Given a program and the top- k retrieved entries, the LLM returns a typed list of findings.

(3) Validator-backed repair loop. Given the program, the union of detector findings, and retrieved knowledge, the LLM proposes a complete patch. The patch is *validated by execution*: it must parse, import, and run to completion, binding the program’s observable result. If it fails, the runtime error is fed back and the model retries up to m times (Algorithm 1). The loop’s internal validator is execution-only: it does *not* see the human-fixed reference, so it directly helps defects that crash but cannot, by itself, certify a silent-semantic fix.

3.2 Repair configurations

Table 1 defines the experimental configurations. Each uses the same fixed detector (Sonnet) and one of the eighteen models as its repair backbone; thus, within a backbone, B0 and B2 differ only in what the pipeline adds, but that addition bundles retrieved entries, findings, and a retry budget of three attempts against B0’s single generation, so we read B2–B0 as a combined-system effect. B1 adds retrieval-grounded findings to the plain B0 prompt, and B2 adds the execution-validator loop, so the B1–B2 gap isolates the validator (RQ4). B2’s prompt,

Table 1. What varies across configurations. Detector is fixed (Sonnet); the repair backbone is each of the 18 models. “Findings” is the static $\cup$ LLM detector union; “asserts clean” is whether the prompt claims no defects when the detector is silent.

Config	Findings	Retr.	Valid.	Asserts “clean”	Debias
B0 plain	—	—	—	—	—
B1 RAG	union	✓	—	if silent	—
B2 naive	union	✓	✓	if silent	—
B3 named-debiased	union	✓	✓	never	named
B4 generic-debiased	union	✓	✓	never	generic

Algorithm 1: Validator-backed repair (B2–B4)

Input: program x , findings F , knowledge base K , budget m , debias flag b

```

1  $D \leftarrow \text{RETRIEVE}(K, x \oplus F, k);$ 
2  $h \leftarrow \emptyset;$  // validator feedback history
3 for  $i \leftarrow 1$  to  $m$  do
4    $\hat{x} \leftarrow \text{LLM}(x, \text{FRAME}(F, b), D, h);$ 
5    $o \leftarrow \text{EXECUTE}(\hat{x});$  // parse, import, run to completion
6   if  $o.ok$  then return  $\hat{x};$ 
7    $h \leftarrow o.error;$ 
8 return  $\hat{x};$ 

```

when nothing is flagged, asserts “static analysis reported no structural defects,” the phrasing we later find *anchors* capable models. B3 and B4 probe whether prompt design alone undoes that anchoring: B3 *enumerates* the silent failure modes it might miss (wrong/forgotten gate, swapped control/target, degrees-vs-radians, endianness), B4 gives only a *generic* incompleteness caveat without family names. The B4–B3 comparison distinguishes general prior correction from leakage of the benchmark taxonomy into the prompt. B4 also represents the deployable control, because it carries no benchmark-specific family hints; a tool may legitimately enumerate known defect classes, but B3’s list is drawn from this benchmark’s own families.

3.3 Semantic oracle (evaluation only)

To *measure* whether a patch is correct we compare its observable output to the human-fixed reference: total-variation distance (TVD) between measurement distributions (≤ 0.08 at 8192 shots with fixed seeds) for sampling programs, and state fidelity (≥ 0.99) for measurement-free circuits. This oracle is used only for evaluation; LINTREPAIR-RAG-Q never consults the reference at repair time.

4 A Two-Factor Framework

Figure 3 presents a two-factor framework for interpreting the results. Propositions 1–2 state two exact claims; the remaining claims are empirical hypotheses.

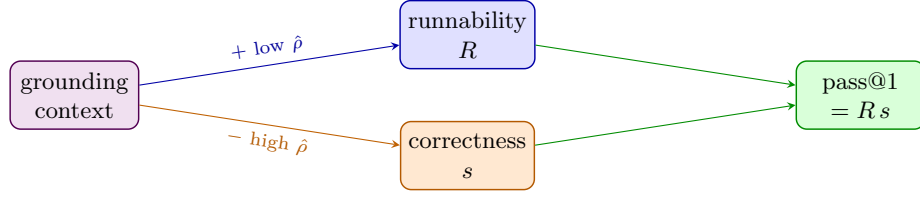


Fig. 3. The two-factor descriptive decomposition: grounding context raises post-validator *runnability* (decisive at low capability $\hat{\rho}$) but can lower *correctness* via interference (at high $\hat{\rho}$); pass@1 is their product, so the sign of grounding’s effect flips with $\hat{\rho}$.

The evaluation supports the first hypothesis and supports the second only conditionally.

4.1 Detection composes (exact)

Model a detector d by the set $\mathcal{R}_d$ of defective instances it flags (true positives) and the set $\mathcal{F}_d$ of clean findings (false positives).

Proposition 1 (Detection composition). *For the union detector, recall = $\frac{|\mathcal{R}_1 \cup \mathcal{R}_2|}{N} \geq \max_i \text{recall}(d_i)$ and $|\mathcal{F}_1 \cup \mathcal{F}_2| \leq |\mathcal{F}_1| + |\mathcal{F}_2|$; if every component is perfectly precise, so is the union.*

For fixed marginal recalls, the gain is largest when the recall sets are disjoint, as expected for a structural static analyser and an LLM that can infer silent semantic intent (tested in RQ1).

4.2 Repair factors into runnability and correctness

Score a repair attempt by the execution oracle. Let ρ be the probability a single sample *runs* to completion and s the probability it is *semantically correct given that it runs*; then $\text{pass@1} = \rho s$. We estimate ρ per model by the *unguided* single-shot run-rate $\hat{\rho}$, a cheap index of *current-SDK* competence rather than general model capability; $\hat{\rho}$ is our capability axis throughout, with its limits discussed in §8. An m -step validator loop replaces ρ by the runnability $R(\rho, m)$:

Proposition 2 (Validator runnability). *Under independent retries, $R(\rho, m) = 1 - (1 - \rho)^m$, and the loop’s gain $g(\rho) = R(\rho, m) - \rho$ satisfies $g(1) = 0$ and increases as ρ decreases in the high-reliability regime ($g'(\rho) = m(1 - \rho)^{m-1} - 1 < 0$ near $\rho=1$).*

Our implemented loop feeds each runtime error into the next prompt, so its retries are *adaptive*, not iid; we use $R(\rho, m)$ as an iid *reference* model and test its qualitative prediction—value scaling with $1 - \rho$ —empirically in RQ4. The loop thus provides little benefit when the initial patch already runs and more as base-model reliability decreases.

4.3 Grounding moves both factors—hence a crossover

Grounding (findings + retrieval) changes both factors in opposite directions. We take as a scope-bounded *assumption*, not a theorem, that grounding raises runnability for *API-crash* defects, $R_g \geq R_u$: retrieved migration knowledge lets a model lacking the current SDK emit APIs that run (§6 confirms this for the API-removal families but not where no API is missing). Grounding may *lower* the correctness factor through anchoring: a report of “no defects” supplies evidence that lowers the model’s prior that a silent defect exists, so $s_g \leq s_u$ for defects outside the analyser’s scope. The net effect of grounding on pass@1 is the exact identity

$$\Delta = R_g s_g - R_u s_u = \underbrace{(R_g - R_u) s_u}_{\text{runnability gain}} + \underbrace{R_g (s_g - s_u)}_{\text{anchoring loss}}. \quad (1)$$

Hypothesis 1 (Crossover). *When the base model is weak (R_u small), the runnability gain dominates and grounding helps; when it is strong ($R_u \approx R_g \approx 1$), the first term vanishes and $\Delta \approx s_g - s_u \leq 0$, so grounding hurts.*

Hypothesis 2 (Debiasing recovers s). *A debiasing prompt that does not assert cleanliness restores s_g toward s_u without leaking which defects to look for.*

4.4 Expected empirical pattern

Section 6 evaluates these claims. The data support the crossover hypothesis across eighteen models. The debiasing hypothesis holds only conditionally: a generic prompt recovers s for high-capability models affected by anchoring (fully for Sonnet), but incurs a small pooled cost. Its value therefore depends on model capability, as does the value of grounding.

5 Implementation and the Q-Defects40 Benchmark

Our study rests on two artifacts: the benchmark we repair against and the toolchain that does the repairing. We describe Q-DEFECTS40 first, then the concrete LINTREPAIR-RAG-Q pipeline and the execution oracle that scores every patch.

5.1 The Q-Defects40 benchmark

Evaluating detection and repair requires programs with known defects and fixes, together with an automated correctness criterion. We constructed Q-DEFECTS40, a set of 40 self-contained Qiskit 2.x programs—32 defective and 8 clean. The 32 defects span eight families (four instances each; Table 2) drawn from documented real-world quantum-software faults: the Qiskit 1.0 API removals that broke existing code [12], structural defects in the LintQ/Smelly-Eight tradition [25,5], and silent-semantic faults in the spirit of Bugs4Q [42].

Table 2. The eight Q-DEFECTS40 families (4 instances each). Source: **M**=Qiskit 1.0 migration guide, **L**=LintQ/Smelly-Eight, **B**=Bugs4Q-style silent bugs. Each is a minimal authored reproduction accepted only after the oracle ($\text{TVD} \leq 0.08$ / $\text{fidelity} \geq 0.99$) confirmed buggy $\neq$ fixed. “Silent”=runs but wrong.

Family	Type	Src	Example	Static?
deprecated-execute-api	crash	M	removed <code>execute()</code>	yes
removed-aer-import	crash	M	<code>qiskit.Aer</code> moved	yes
deprecated-bind-parameters	crash	M	<code>bind_</code> → <code>assign_</code>	yes
qubit-index-oob	crash	L	gate on absent qubit	yes
clbit-size-mismatch	crash	L	register too small	yes
missing-measurement	crash/sil.	L	counts, no measure	partial
wrong-gate-or-param	silent	B	<code>cz</code> for <code>cx</code>	no
endianness-bit-order	silent	B	little-endian misread	no

Each defective program is paired with a minimal human-written fix. The eight families and the four-instances-each budget were fixed *a priori* from these documented sources before evaluating any model; thus, the taxonomy was not tuned to the results. Per-family repair outcomes are analysed in §6 and released in full. The eight clean programs are correct reference idioms; together with the 32 human fixes they form a 40-program negative set (§6) for measuring false positives.

Every program binds an observable result on a successful run (a counts dictionary, or a statevector for measurement-free circuits). Correctness is decided by the execution oracle of §3 ($\text{TVD} \leq 0.08$ at 8192 shots, or $\text{fidelity} \geq 0.99$). We verified the benchmark by execution. A build script confirms that every crash defect raises an exception, every semantic defect runs but diverges from its fix (median TVD 0.50), and every fix and clean program runs cleanly. We rejected one proposed “endianness” instance because it computed the correct value. We release the full construction log (sources, accept/reject decisions, checks) so the benchmark is auditable.

5.2 LintRepair-RAG-Q implementation

LINTREPAIR-RAG-Q is implemented in Python on Qiskit 2.4.2 and Qiskit-Aer 0.17.2. The static detector is a ~ 200 -line AST/pattern analyser. The knowledge base holds 15 general defect-family documents (no instance answers) indexed by a dependency-free TF-IDF retriever; we retrieve the top $k=4$ documents. The execution validator runs each candidate in an isolated subprocess (60 s timeout) and reports the terminating error, if any, to the repair loop.

Models. To assess generality and capability dependence, we evaluate eighteen repair backbones spanning proprietary APIs and open weights from 3B to 72B, over a wide capability range: GPT-4o and GPT-4o-mini (OpenAI), Gemini 2.5 Pro, Flash, and Flash-Lite (Google), Claude Haiku 4.5 and Sonnet 4.6 (Anthropic), and eleven open-weight models—Qwen2.5 (7B/14B/72B), Qwen2.5-Coder (7B/32B), Llama-3.1-8B, Llama-3.2-3B, Llama-3.3-70B,

Gemma2-9B, Mistral-7B, and Command-R-35B—run locally via Ollama.⁴ Across these the unguided single-shot run rate $\hat{p}$ ranges from 0.00 to 1.00. For every model the *same* backbone serves the unguided baseline and the grounded system, so within a backbone only the pipeline’s additions differ, which bundle retrieved entries, findings, and a three-attempt retry budget against B0’s single generation. The retrieval-grounded *detector* is fixed to one model (Sonnet) across all repair experiments, so detection findings are identical regardless of the repair backbone. Decoding is greedy (temperature 0); the validator budget is $m=3$. All API calls are cached on disk keyed by their full input, so the reported analysis reproduces from frozen outputs, though identical regeneration from the providers is not guaranteed.

Metrics and statistics. Detection recall is program-level (of the 32 defective programs, the fraction whose true family is flagged) and precision is $TP/(TP+FP)$, evaluated over the expanded 40-program negative set (the 8 clean idioms plus the 32 human-fixed programs), for which we also report the raw false-positive count. Repair pass@1 is the oracle-correct fraction of the final patch from one repair invocation over the 32 defective programs; B0 permits a single generation, whereas B2–B4 permit up to three attempts inside the validator loop. We report Wilson 95% intervals and compare configurations on the *same* instances with a paired bootstrap (10^4 resamples) *and* the exact McNemar test, reported together; with $n=32$ we treat single-model differences as descriptive and reserve significance claims for the higher-powered pooled tests (§8).

6 Evaluation

We evaluate the four research questions on Q-DEFECTS40 across eighteen models and test the propositions and hypotheses from §4. A single driver produces all results from program executions. We report paired bootstrap confidence intervals with exact McNemar p -values and claim significance only when both tests agree.

6.1 RQ1: Does grounding help detection?

Table 3 reports recall and precision. The static analyser reaches recall 0.719, the retrieval-grounded LLM detector 0.781, and their union 0.812. All three produce no false positives on an *expanded matched negative set* of 40: the 8 clean idioms plus the 32 human-fixed versions of the defective programs, structurally identical to the defects except for the fix. Precision is thus 1.000 on 40 negatives, 32 defect-matched (though 40 negatives cannot establish zero false positives *beyond* Q-DEFECTS40). The recall gain lands where Proposition 1 predicts: on silent-semantic families the static analyser is blind (0/4 on endianness and wrong-gate) while the LLM recovers 2/4 and 1/4, and the union loses nothing static found.

⁴ Exact API identifiers (`claude-sonnet-4-6`, `gpt-4o`, `gemini-2.5-pro`, ...), Ollama tags, and cached responses are recorded in the artifact; call dates and hosted-model revisions were not recorded.

Table 3. RQ1 Detection on Q-DEFECTS40 over 32 defective programs. N^- is the negative set each detector was scored against: our detectors use all 40 matched negatives (8 clean idioms and 32 human fixes), whereas LintQ was run on the 8 clean programs only, so the precision columns are not a common-set comparison. “n/a” marks a detector that emits no positives.

Detector	N^-	Precision	Recall	F1
LintQ, high-prec. (default) [25]	8	n/a	0.000	n/a
LintQ, all checkers [25]	8	0.786	0.344	0.478
Static (rule-based)	40	1.000	0.719	0.836
RAG-LLM	40	1.000	0.781	0.877
Union (LINTREPAIR-RAG-Q)	40	1.000	0.812	0.897

Robustness to a much stronger or differently-phrased detector is an open threat (§8), as the detector is fixed to one model.

Head-to-head with LintQ. We ran LintQ [25], the state-of-the-art rule-based Qiskit analyser (CodeQL 2.11.2), on Q-DEFECTS40. It models our circuits (its `CircuitSize` query reports qubit/bit counts), so non-detection reflects rule coverage, not a parse failure. At its high-precision setting it flags *none* of the 32 defects (recall 0.000): its catalogue targets measurement/composition/transpilation anti-patterns orthogonal to our families. All checkers enabled raises recall to 0.344 at precision 0.786, but the hits are *incidental* (two structural checkers fire on endianness-buggy and endianness-*correct* programs alike). LINTREPAIR-RAG-Q’s union reaches 0.812 at precision 1.000—a difference of scope, not general superiority: a rule set built for one defect distribution does not transfer to the migration-crash and silent-semantic families dominating Qiskit-2.x.

6.2 RQ2: How does grounding affect repair across models?

Table 4 reports the central result. Pooled over all eighteen models on matched instances (576 paired programs), grounding (B2) beats unguided repair (B0) by +0.160 (CI [0.109, 0.214], McNemar $p < 0.001$; 168 vs 76 wins). Because the same 32 programs recur across models, we also estimate it *without* assuming instance independence: a program-clustered bootstrap gives +0.160, CI [0.038, 0.276], and a mixed-effects logistic (random intercepts for program and model) an odds ratio of 2.69 (CI [2.2, 3.3]). Both exclude the null; the clustered interval is merely wider, so correlated instances do not explain the effect.

The pooled mean masks a crossover governed by base capability $\hat{\rho}$: the unguided single-shot *run*-rate (fraction of B0 patches that execute), a cheap proxy for how current the model’s SDK knowledge is. It ranges from 0 (GPT-4o-mini) to 1 (Sonnet) (Fig. 4; Table 4 sorted by $\hat{\rho}$). Lower-capability models gain substantially: grounding lifts GPT-4o-mini by +0.625 (0% $\rightarrow$ 63%), Qwen2.5-Coder-32B and GPT-4o by +0.438, Qwen2.5-7B by +0.312, Llama-3.1-8B by +0.281, and Gemma2-9B by +0.219. GPT-4o-mini is extreme: *every* unguided patch fails to run because it emits removed Qiskit-1.0 APIs (`execute`, `qiskit.Aer`, `assemble`); retrieved migration knowledge makes its patches run and correct.

For the strongest models, grounding reduces accuracy. Sonnet falls $0.969 \rightarrow 0.875$ and Haiku $0.781 \rightarrow 0.688$ (-9.4 points each, $-3/32$); descriptive at $n=32$, but consistent across both and confirmed at scale below. The strongest *open* model, Qwen2.5-72B, also turns negative ($0.656 \rightarrow 0.625$), so anchoring is not confined to proprietary frontier models. The loss concentrates on silent-semantic instances: Sonnet’s wrong-gate repair falls from $4/4$ (B0) to $1/4$ (B2), where no detector fires and the model preserves a bug it fixes unprompted (Hypothesis 1; cause dissected in §6.4). The trend is not strictly monotonic—two mid-range outliers stall or backfire (Gemini 2.5 Pro $+0.094$; Llama-3.2-3B -0.094)—but the sign pattern holds across the range, and two pooled analyses make the crossover a *measured* effect, not a visual one.

Stratifying by defect kind sharpens this. Splitting the 32 programs into the 9 silent-semantic instances our static rules do not flag (four wrong-gate, four endianness, one runs-but-wrong missing-measurement) and the other 23 crash instances: grounding helps *crash* defects by $+0.258$ (CI $[0.196, 0.321]$, $p < 0.001$; $n=414$) but *hurts* silent-semantic ones by -0.093 (CI $[-0.173, -0.012]$, $p = 0.036$; $n=162$). This last figure sits near the 0.05 boundary and is *not* Bonferroni-robust across our ~ 7 tests, so we lean on the powered $n=115$ set (§6.4), which corroborates its *direction* decisively, rather than the p -value alone. Grounding thus mainly helps patches run under current APIs, where retrieval supplies missing knowledge, but reduces accuracy on silent semantic defects.

Across models, capability predicts the grounding gain (Pearson $r = -0.629$, Spearman -0.538 , $n=18$): the more capable the model, the smaller or more negative the gain. We take this capability $\times$ grounding *interaction* as our primary estimand. A model-level regression of the gain on $\hat{p}$ has slope -0.47 (bootstrap CI $[-0.68, -0.19]$, $R^2=0.40$), so each unit of capability costs ≈ 0.47 of gain; the fit falls from $+0.40$ at $\hat{p}=0$ to -0.07 at $\hat{p}=1$, crossing zero near $\hat{p} \approx 0.84$ (Fig. 4). This interaction, not the pooled $+0.16$, is the primary effect. Part of the coupling is arithmetic—the gain cannot exceed $1-B0$ and capable models have less headroom (headroom correlates $+0.57$ with the gain)—but the capability effect *survives* it: the partial correlation controlling for headroom is still -0.33 , and the strongest models drop in *absolute* terms (Sonnet $42 \rightarrow 0$ on the powered silent set, §6.4), which a ceiling alone cannot explain.

A per-family breakdown (pooled over the 18 models) localises the effect. B2 lifts every API-removal crash family (removed-aer $0.32 \rightarrow 0.81$, deprecated-execute $0.32 \rightarrow 0.72$, deprecated-bind $0.06 \rightarrow 0.65$) but not the silent-semantic ones (wrong-gate $0.28 \rightarrow 0.13$, endianness flat), nor the two non-API crash families (missing-measurement, qubit-index-oob), where no API is missing. The discordant pairs agree: of grounding’s 168 wins, 153 are crash defects and only 15 silent-semantic, which instead account for 30 of the 76 losses; 48 of those losses, including 22 of the 30 silent-semantic ones, fall on high-capability models ($\hat{p} \geq 0.5$). Together these support Hypothesis 1 beyond per-model anecdote.

Is grounding even needed for the crash families? A deterministic Qiskit-1.0 migration script (no LLM) fixes only 10/32 defects (all within the API-removal

Table 4. RQ2–RQ3 Repair pass@1 by model (32 defective programs), sorted by $\hat{\rho}$, the fraction of B0 patches that execute. B0 plain, B2 naive-grounded, B3 family-specific warning, B4 generic warning; the last column is B2 – B0. Grounding helps weak models and hurts strong ones.

Model	$\hat{\rho}$	B0	B2	B3	B4	B2–B0
GPT-4o-mini	0.00	0.000	0.625	0.719	0.625	+0.625
Qwen2.5-14B	0.12	0.125	0.406	0.344	0.312	+0.281
Mistral-7B	0.16	0.031	0.156	0.156	0.031	+0.125
Llama-3.1-8B	0.19	0.094	0.375	0.438	0.375	+0.281
Gemini 2.5 Pro	0.41	0.344	0.438	0.250	0.219	+0.094
Llama-3.2-3B	0.44	0.156	0.062	0.094	0.000	−0.094
Gemma2-9B	0.47	0.156	0.375	0.344	0.375	+0.219
Qwen2.5-7B	0.47	0.188	0.500	0.375	0.406	+0.312
Command-R-35B	0.47	0.219	0.281	0.312	0.281	+0.062
GPT-4o	0.50	0.406	0.844	0.844	0.812	+0.438
Qwen2.5-Coder-7B	0.53	0.250	0.250	0.219	0.156	+0.000
Qwen2.5-Coder-32B	0.56	0.250	0.688	0.719	0.688	+0.438
Llama-3.3-70B	0.59	0.406	0.500	0.594	0.500	+0.094
Gemini Flash-Lite	0.69	0.500	0.656	0.688	0.562	+0.156
Gemini Flash	0.78	0.656	0.719	0.688	0.750	+0.062
Qwen2.5-72B	0.84	0.656	0.625	0.750	0.719	−0.031
Haiku 4.5	0.88	0.781	0.688	0.812	0.750	−0.094
Sonnet 4.6	1.00	0.969	0.875	0.938	0.969	−0.094
Pooled B2–B0: +0.160 [.109,.214], $p < 0.001$, 168/76						

families; 0/4 on every silent-semantic family)—so grounding’s weak-model benefit is broader than a codemod, while *neither* touches the silent-semantic defects.

6.3 RQ3: Can debiasing undo the anchoring?

Only conditionally; the value of debiasing is itself capability-dependent. The named debiasing B3, which tells the model the analyser may miss “a wrong gate, a swapped control, degrees-vs-radians, an endianness error,” gives a small positive difference over naive grounding that is not statistically significant pooled across the eighteen models (B3–B2 = +0.012, $p = 0.48$; 39 wins, 32 losses)—and because those four modes are the benchmark’s silent families, any advantage is answer-key leakage. The deployable generic B4, an “analyser may be incomplete” caveat without family names, modestly reduces naive grounding (B4–B2 = −0.030, CI [−0.056, −0.003], $p = 0.04$; 21 wins, 38 losses); like the silent-semantic harm this sits near the 0.05 boundary and is not Bonferroni-robust, so we read it as a second underlying crossover rather than a standalone effect.

At the model level ($n=32$), B4 recovers the anchoring loss on the two strongest models (Sonnet $0.875 \rightarrow 0.969$, restoring the full 9.4 points, and Haiku $0.688 \rightarrow 0.750$) but *erodes* the grounding benefit on weaker models that depend on it (Mistral-7B -0.125 , Qwen2.5-7B -0.094 , Gemini 2.5 Pro -0.219). B3 still edges B4 (pooled B3–B4 = +0.042, $p < 0.001$; 37 vs 13), a gap we

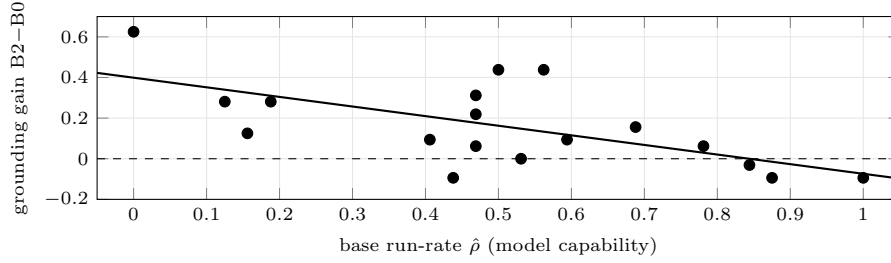


Fig. 4. The crossover: grounding’s repair gain (B2–B0) is large at low capability and turns negative (anchoring) as $\hat{\rho}$ rises. Points are the eighteen models ($n=32$ each); the line is the fitted interaction (slope -0.47 , $R^2=0.40$), crossing zero at $\hat{\rho} \approx 0.84$.

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cz(0, 1)      # BUG: cz where cx (CNOT) is intended
# FIX: qc.cx(0, 1) -> restores the Bell state
qc.measure([0, 1], [0, 1])
counts = run(qc)  # buggy -> {'00', '10'}; qubit 1 never flips
```

Fig. 5. Case study F1-cz-for-cx: a silent wrong-gate bug that no analyser flags. Sonnet repairs it unprompted (B0) but leaves it under grounding (B2).

attribute to taxonomy leakage, not prior-correction. This qualifies Hypothesis 2: prompt-level debiasing is no universal remedy—it reduces anchoring on capable models but harms weak ones, the named variant edges the generic one only by naming the benchmark’s families, and even B3 does not significantly beat naive grounding; no variant is uniformly beneficial.

6.4 What causes the strong-model regression? A controlled dissection

A case study. Fig. 5 shows F1-cz-for-cx, a two-qubit Bell preparation applying cz where cx (CNOT) is intended, so entanglement is lost and measurement returns $\{00, 01\}$ (keys written q_1q_0) rather than the Bell support $\{00, 11\}$. No detector fires (empty union finding): cz is a valid call with no syntactic signature. Asked plainly (B0), Sonnet restores cx; given the *grounded* prompt (B2)—the “no structural defects” report plus retrieved API docs—the same model accepts the clean verdict and leaves cz in place. Grounding suppresses a repair the model makes unprompted; we dissect the cause next.

We first attributed the regression to anchoring on the cleanliness claim; matched controls on the two affected models over the nine silent-semantic instances indicate a broader cause. It is not a decoding artifact: under pass@5, unguided Sonnet still fixes 9/9 silent bugs to grounded’s 6/9, and on the full 32 the gap persists (31/32 vs 28/32). Nor is it specific to the “no defects” string—a length- and retrieval-matched control (B0filler: the same documents as neutral background, no findings or cleanliness claim) reproduces almost the entire drop

(Sonnet $9 \rightarrow 6$, Haiku $7 \rightarrow 3$), and irrelevant documents regress too. The harm tracks the presence of grounding context, consistent with distraction by added or noisy context, a documented LLM failure mode [30,17,8,39]. The only intervention that helps redirects verification: the generic “verify independently” caveat B4 ($6 \rightarrow 8$ Sonnet, $4 \rightarrow 6$ Haiku). We describe the mechanism as grounding-context interference, the cleanliness assertion a contributing but not isolable cause, confirmed at scale below.

Scale check ($n=115$, *eighteen models*). To rule out small-sample effects, we generated 115 silent bugs by *mutating the correct programs* (gate/control/angle perturbations, all execution-validated) and re-ran B0/B2/B0filler on all eighteen models. The crossover becomes nearly universal: grounding’s silent-bug benefit (#correct of 115) is positive for only GPT-4o-mini (at the unguided floor, $1 \rightarrow 13$) and zero or negative for the other seventeen, the loss growing with base skill up to Sonnet’s $42 \rightarrow 0$ (all 42 discordant pairs losses, $p \ll 10^{-3}$); it also occurs in open weights (Llama-3.1-8B $25 \rightarrow 1$, Llama-3.3-70B $21 \rightarrow 0$). The length-matched B0filler shows the mechanism varies by model: ≈ 0 for the open models (pure context-distraction), equal to B0 for Gemini 2.5 Pro (14; pure grounding *content*), and between for Sonnet (22; both). The 115 mutants derive from 28 distinct parent programs, so they broaden coverage of a synthetic semantic task rather than adding independent real-world evidence; they support the defect-type effect at scale but do not independently validate the competence-dependent mechanism.

Retrieval budget. A separate top- k sweep shows the optimal budget varies with capability: the most-anchored model (Sonnet) does best with the *smallest* budget (29/32 at $k=2$ vs 27–28 at $k \in \{4, 6\}$; less context, less distraction), while the most-rescued weak model (GPT-4o-mini) improves *monotonically* with more ($18 \rightarrow 20 \rightarrow 23$). Swapping the retriever for BM25 barely moves results, so budget matters more than retriever choice.

6.5 RQ4: When does the validator loop earn its keep?

Proposition 2 predicts the loop is near-inert at high $\hat{\rho}$ and more active as the base model weakens; the data follow (full traces in the artifact). The runnability gain arrives two ways: pure *grounding* (GPT-4o-mini’s exec rate jumps $0.00 \rightarrow 1.00$ with *zero* loop iterations) and the *loop* (Llama-3.1-8B retried 11/32 patches to 0.78). Isolating the validator (B2 vs B1, API models) confirms it is inert when grounded first-shots already run: Sonnet and GPT-4o-mini have B1=B2 (28/32, 20/32). At the bottom the loop retries most but cannot rescue a weak base (Mistral-7B retried 30/32 to only 0.44 exec), so it supplements rather than replaces base capability. Its marginal value scales with $1 - \hat{\rho}$; budget fixed at $m=3$ (mean iters ≤ 2.7).

6.6 Replication on independently-sourced real bugs

Because Q-DEFECTS40 is author-curated, we test the crossover on Bugs4Q [42], real Qiskit bugs from issue trackers. API drift is severe (only 2/42 run under

Qiskit 2.4.2 vs 19/42 under the native Qiskit 0.45, a reproducibility gap that itself motivates a current-SDK benchmark). On the 12 bugs that reproduce deterministically (four models), both parts replicate directionally: the validator rescues crash bugs ($7 \rightarrow 12$ of 28), while on *silent* bugs naive grounding *anchors* the two strongest models exactly as on Q-DEFECTS40 ($4/10 \rightarrow 0/10$). The weak-model rescue is muted (our KB is Qiskit 2.x, these bugs 0.x) so the pooled effect is neutral—but the mechanism appears on independently curated bugs, not only our families.

6.7 Cost

Repair issues a single model call in the common case (mean latency 3.6 s Sonnet, 2.0 s Haiku; open weights 1–20 s/call locally). The complete eighteen-model study used a few thousand calls and reproduces from cache at zero marginal cost.

7 Discussion and Implications

Capability-gated deployment. The crossover suggests a practical policy for LLM repair in quantum and other rapidly evolving software: estimate capability from the unguided run rate $\hat{\rho}$ on a few examples; at low $\hat{\rho}$ ground and validate, but at high $\hat{\rho}$ withhold the analyser’s cleanliness verdict (or add a generic “verify independently” caveat) and avoid enumerating defect families drawn from the evaluation set, which advantages the prompt on that benchmark. An incomplete analyser is not a neutral input: its verdict can reduce a capable model’s recall on defects outside its scope, so grounding channels need explicit coverage caveats.

A control for debiasing claims. The only prompt variant that improves repair across models enumerates the silent defect families, injecting benchmark information rather than correcting a prior; our B3–B4 ablation measures this leakage. Evaluations of debiasing or self-verification should include a control that does not encode the benchmark’s taxonomy—the anchoring itself is consistent with documented framing sensitivity [29,35].

Validation as a safety net. The validator loop never fired on our most capable models and fired increasingly as the base weakened (recovering a third of Llama-3.1-8B’s crashing patches): little overhead for strong models, more for weak ones. Reporting the unguided B0 baseline and an oracle separating runnability from correctness was essential to see that grounding *regresses* on strong models.

8 Threats to Validity

Construct. Correctness is decided by an execution oracle (TVD over counts; state fidelity; thresholds 0.08/0.99, not tuned). The human fix matches the reference at TVD= 0 (median and max) while buggy outputs sit at $\text{TVD} \geq 0.23$ (median 0.50), so labels are *invariant* for every threshold in $[0.02, 0.20]$, and the counts-based human fixes reproduced their outputs across five executions at the

benchmark’s fixed seeds, which shows repeatability rather than seed robustness. The oracle therefore distinguishes each fix from its corresponding bug. An audit shows every flagged program names its true family (static 23/23, LLM 25/25, union 26/26; 0 false positives), ruling out wrong-family hits, though not the LLM detector’s wrong-rationale (a human explanation study is future work).

Internal. The knowledge base and prompts are ours; we limit bias by keeping the KB general (no instance answers), reporting the unguided B0 and static-only baselines (which get no prompt engineering), and controlling the named debiasing B3 against the generic B4. Decoding is greedy and all calls cached, so the analysis replays deterministically from frozen outputs.

External. Eighteen models spanning $\hat{p} \in [0, 1]$ show that anchoring is not confined to proprietary models (it recurs in open weights, §6); but the benchmark is Qiskit-only, the 32 defects are curated reproductions rather than wild-mined bugs, the programs are short, and the detector is a single model (Sonnet). We replicated on independently-sourced real bugs (Bugs4Q, §6.6): strong-model silent anchoring reappears ($4/10 \rightarrow 0/10$) on bugs we did not design, directional at $n=12$. Mining public repositories yielded only shallow crash/import bugs, which itself motivates a curated, current-SDK benchmark. A cross-SDK probe on 10 Cirq programs did not reproduce the interference—capable models repair at ceiling (Sonnet and Haiku 10/10)—consistent with the effect living in a narrow difficulty band that short Cirq programs sit above. We *hypothesize* the mechanism is SDK-agnostic but do not claim it: testing needs a realistic cross-SDK corpus of comparable difficulty, not currently available.

Statistical conclusion. With 32 defects per model, the McNemar discordant-pair floor (≥ 6 same-direction pairs for $p < 0.05$) does not make every per-model test null: six grounding improvements are individually significant by exact McNemar, three of them (GPT-4o-mini, GPT-4o, Qwen2.5-Coder-32B) under a Bonferroni threshold of 0.05/18, whereas the strong-model regressions are not (Sonnet $p = 0.38$, Haiku $p = 0.55$). We nonetheless treat single-model numbers as descriptive and claim significance only for pooled comparisons where the bootstrap CI excludes zero *and* exact McNemar agrees: B2–B0 ($p < 0.001$) and the B3–B4 leakage gap (CI [0.019, 0.066], $p < 0.001$). The strong-model anchoring (≈ -9 points) is consistent with the proposed mechanism but not significant at $n=32$; two pooled claims near $p \approx 0.04$ (silent-semantic harm; B4 erosion) do not survive Bonferroni and are corroborated—not established—by the powered $n=115$ set. A larger, in-the-wild benchmark is the principal limitation.

9 Conclusion

Across eighteen models, grounding’s value in LLM quantum-program repair is a capability-dependent crossover: it improves detection (recall $0.72 \rightarrow 0.81$) and rescues weak repair models (pooled $+16.0$, $p < 0.001$; GPT-4o-mini $0 \rightarrow 63\%$) but anchors the strongest on silent defects (-9.4 points), in proprietary and open weights alike. Prompt-level mitigation is likewise capability-dependent, and the variant that edges ahead only leaks the benchmark taxonomy (B3–B4) without

beating naive grounding. These motivate a capability-gated deployment policy (§7). We *hypothesize* the mechanism is not SDK-specific, but a cross-SDK Cirq probe did *not* reproduce it (§8); whether it and the thresholds transfer beyond Q-DEFECTS40 is open.

Data availability. Q-DEFECTS40, LINTREPAIR-RAG-Q, the response cache, and scripts that regenerate every reported number are provided as supplementary material and will be publicly archived.

References

1. Bouzenia, I., Devanbu, P., Pradel, M.: RepairAgent: An autonomous, LLM-based agent for program repair. In: ICSE (2025).
2. Cai, Y., Hou, Z., Sanan, D., Luan, X., Lin, Y., Sun, J., Dong, J.S.: Automated program refinement: Guide and verify code large language model with refinement calculus. *Proc. ACM Program. Lang.* **9**(POPL), 2057–2089 (2025).
3. Cassieri, P., Scanniello, G., Shin, S.Y., Pastore, F., Bianculli, D.: Beyond rules: LLM-powered linting for quantum programs. arXiv:2605.03943 (2026).
4. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code (2021).
5. Chen, Q., Câmara, R., Campos, J., Souto, A., Ahmed, I.: The smelly eight: An empirical study on the prevalence of code smells in quantum computing. In: *Proc. ICSE*. pp. 358–370 (2023).
6. Chen, X., Lin, M., Schärli, N., Zhou, D.: Teaching large language models to self-debug. In: *ICLR* (2024)
7. Cross, A.W., Javadi-Abhari, A., Alexander, T., de Beaudrap, N., Bishop, L.S., Heidel, S., Ryan, C.A., Sivarajah, P., Smolin, J., Gambetta, J.M., Johnson, B.R.: OpenQASM 3: A broader and deeper quantum assembly language. *ACM Trans. on Quantum Computing (TQC)* **3**(3) (2022).
8. Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonello, N., Silvestri, F.: The power of noise: Redefining retrieval for RAG systems. In: *SIGIR* (2024)
9. De Stefano, M., Pecorelli, F., Di Nucci, D., Palomba, F., De Lucia, A.: Software engineering for quantum programming: How far are we? *Journal of Systems and Software* **190**, 111326 (2022).
10. Guo, X., Zhao, J., Zhao, P.: On repairing quantum programs using ChatGPT. In: *Int. Workshop on Quantum Software Engineering (Q-SE)*. pp. 9–16 (2024).
11. Huang, J., Chen, X., Mishra, S., Zheng, H.S., Yu, A.W., Song, X., Zhou, D.: Large language models cannot self-correct reasoning yet. In: *ICLR* (2024)
12. Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C.J., Lishman, J., Gacon, J., Martiel, S., Nation, P.D., Bishop, L.S., Cross, A.W., Johnson, B.R., Gambetta, J.M.: Quantum computing with Qiskit (2024).
13. Jin, M., Shahriar, S., Tufano, M., Shi, X., Lu, S., Sundaresan, N., Svyatkovskiy, A.: InferFix: End-to-end program repair with LLMs. In: *ESEC/FSE* (2023).
14. Kaye, P., Laflamme, R., Mosca, M.: *An Introduction to Quantum Computing*. Oxford University Press (2007)
15. Le, X.B., Lin, S.W., Sun, J., Sanan, D.: A quantum interpretation of separating conjunction for local reasoning of quantum programs based on separation logic. *Proc. ACM Program. Lang.* **6**(POPL), 1–27 (2022).

16. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: *Proc. NeurIPS*. vol. 33, pp. 9459–9474 (2020)
17. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the ACL (TACL)* (2024)
18. Liu, S., Bai, G., Utting, M., Yang, G.: RelRepair: Enhancing automated program repair by retrieving relevant code. *arXiv:2509.16701* (2025)
19. Liu, Z., Du, X., Liu, H.: ReAPR: Automatic program repair via retrieval-augmented large language models. *Software Quality Journal* **33**(3), 30 (2025).
20. Mukherjee, M., Hellendoorn, V.J.: SOSecure: Safer code generation with RAG and Stack Overflow discussions. *arXiv:2503.13654* (2025)
21. Murillo, J.M., Garcia-Alonso, J., Moguel, E., Barzen, J., Leymann, F., Ali, S., Yue, T., Arcaini, P., Pérez-Castillo, R., de Guzmán, I.G.R., Piattini, M., Ruiz-Cortés, A., Brogi, A., Zhao, J., Miranskyy, A., Wimmer, M.: Quantum software engineering: Roadmap and challenges ahead. *ACM Trans. on Software Engineering and Methodology (TOSEM)* **34**(5) (2025).
22. Nielsen, M.A., Chuang, I.L.: *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edn. (2010)
23. Olausson, T.X., Inala, J.P., Wang, C., Gao, J., Solar-Lezama, A.: Is self-repair a silver bullet for code generation? In: *ICLR* (2024)
24. Paltenghi, M., Pradel, M.: MorphQ: Metamorphic testing of the Qiskit quantum computing platform. In: *Proc. ICSE*. pp. 2413–2424 (2023).
25. Paltenghi, M., Pradel, M.: Analyzing quantum programs with LintQ: A static analysis framework for Qiskit. *Proc. ACM on Software Engineering (PACMSE)* **1**(FSE), 2144–2166 (2024).
26. Paltenghi, M., Pradel, M.: A survey on testing and analysis of quantum software. *arXiv:2410.00650* (2024).
27. Pham, A.B.B., Nguyen, H.T., Usman, M.: QBugLM: An agentic benchmarking framework for LLM-based quantum software debugging. *arXiv:2606.07314* (2026)
28. Rieffel, E.G., Polak, W.H.: *Quantum Computing: A Gentle Introduction*. MIT Press (2011)
29. Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askill, A., Bowman, S.R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S.R., Kravec, S., et al.: Towards understanding sycophancy in language models. *arXiv:2310.13548* (2023).
30. Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E.H., Schärli, N., Zhou, D.: Large language models can be easily distracted by irrelevant context. In: *ICML* (2023)
31. Shi, J., Xiao, Z., Shi, H., Jiang, Y., Li, X.: QuanTest: Entanglement-guided testing of quantum neural network systems. *ACM Trans. on Software Engineering and Methodology (TOSEM)* (2024).
32. Shin, S.Y., Pastore, F., Bianculli, D.: Quantum program linting with LLMs: Emerging results from a comparative study. In: *Proc. IEEE QCE*. vol. 2, pp. 181–186 (2025)
33. Sun, J., Liu, Y., Dong, J.S., Pang, J.: PAT: Towards flexible verification under fairness. In: *Proc. CAV. LNCS*, vol. 5643, pp. 709–714. Springer (2009).
34. Tan, S., Lu, L., Xiang, D., Chu, T., Lang, C., Chen, J., Hu, X., Yin, J.: HornBro: Homotopy-like method for automated quantum program repair. *Proc. ACM on Software Engineering (PACMSE)* **2**(FSE), 734–756 (2025).

35. Turpin, M., Michael, J., Perez, E., Bowman, S.R.: Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In: Proc. NeurIPS (2023).
36. Wang, J., Zhang, Q., Xu, G.H., Kim, M.: QDiff: Differential testing of quantum software stacks. In: Proc. ASE. pp. 692–704 (2021).
37. Xia, C.S., Wei, Y., Zhang, L.: Automated program repair in the era of large pre-trained language models. In: Proc. ICSE. pp. 1482–1494 (2023).
38. Ye, J., Xia, S., Zhang, F., Arcaini, P., Ma, L., Zhao, J., Ishikawa, F.: QuraTest: Integrating quantum specific features in quantum program testing. In: Proc. ASE. pp. 1149–1161 (2023).
39. Yoran, O., Wolfson, T., Ram, O., Berant, J.: Making retrieval-augmented language models robust to irrelevant context. In: ICLR (2024)
40. Yoshida, C., Ishimoto, Y., Nourry, O., Kondo, M., Matsushita, M., Kamei, Y., Higo, Y.: Leveraging mutation analysis for LLM-based repair of quantum programs. In: Proc. SANER (Early Research Achievements) (2026).
41. Zappin, J., Stalnaker, T., Chaparro, O., Poshyvanyk, D.: When quantum meets classical: Characterizing hybrid quantum-classical issues discussed in developer forums. In: Proc. ICSE. pp. 2931–2943 (2025)
42. Zhao, P., Miao, Z., Lan, S., Zhao, J.: Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs. *Journal of Systems and Software* **205**, 111805 (2023).